PAOSITRA SMART ASSET

Guide de présentation — Socle Mois 1

Version mise à jour : bugs trouvés et corrigés, tests formalisés (12/12), scénario validé par toi-même.

Document personnel de préparation — à réviser, pas à lire mot pour mot.

État réel du projet à ce jour

Partie	État
MCD → PostgreSQL (14 tables)	✅ Fonctionnel, testé
Entrée individualisée (Article + Mouvement)	✅ Fonctionnel, testé automatiquement + toi-même en admin
Entrée consommable (quantité globale)	✅ Fonctionnel, testé automatiquement + toi-même
Stock global TypeArticle (les deux cas)	✅ Fonctionnel — corrigé pendant le projet, voir Étape 5
Sortie + blocage si stock insuffisant	✅ Fonctionnel — corrigé pendant le projet, voir Étape 6
Traçabilité (Mouvement)	✅ Fonctionnel, testé
Admin Django (cohérent avec la logique métier)	✅ Fonctionnel — corrigé, voir Étape 5
API REST (7 ressources + 3 actions métier)	✅ Fonctionnel, testé
Authentification + désactivation réelle d'un compte	✅ Fonctionnel — corrigé cette semaine
12 tests automatisés (python manage.py test)	✅ 12/12 passent
Frontend React, QR codes, Dashboard	❌ NON TERMINÉ — Mois 2/3, comme prévu depuis le début
Workflow complet à 2 signatures (autorisation/validation)	⚠ Modèle prêt (table Intervient), vue API pas encore écrite

Étape 1 — Introduction

2 min

🗣 CE QUE JE DIS

"Bonjour, je m'appelle [ton nom], étudiante en L3 informatique à l'ITU Madagascar. Je vais vous présenter le socle technique de PAOSITRA SMART ASSET, un système de gestion, d'inventaire et de traçabilité des matériels de Paositra Malagasy, développé pendant le premier mois de mon stage."

"Ny tanjon'ity projet ity dia ny hanamora ny fitantanana sy ny fanarahamaso an'ireo matériels ao amin'ny CAME, Centre d'Approvisionnement en Matériels et Équipements."

Étape 2 — Le problème et le périmètre

2 min

🗣 CE QUE JE DIS

"Avant ce projet, le suivi des matériels n'était ni centralisé ni traçable dans le temps. Mon objectif pour ce premier mois n'était pas de tout terminer, mais de construire un socle solide : base de données, logique métier, API — testé et démontrable, pas juste esquissé."

QUESTION DU JURY — "Pourquoi seulement un mois pour ça ?"

RÉPONSE : Le stage dure 3 mois. Le plan de travail prévoyait volontairement de consacrer le premier mois au socle technique (données, règles métier, API), et de réserver les mois 2 et 3 au frontend, aux QR codes, au dashboard et au workflow complet. Construire une base solide avant d'empiler des fonctionnalités évite d'avoir à tout reprendre plus tard.

Étape 3 — Le MCD et les règles métier

3 min

👁 CE QUE JE MONTRE

Ouvre `docs/JOURNAL_DECISIONS.md`, section MCD.

🗣 CE QUE JE DIS

"Ity TYPE_ARTICLE ity dia mamaritra ny karazana ankapobeny, ohatra 'Ordinateur portable'. Ny ARTICLE kosa dia ny exemplaire physique tokana."

"La règle centrale du projet : le stock global d'un type (TypeArticle.quantite_stock) bouge toujours, individualisé ou non. Pour un type individualisé, on crée en plus un Article par exemplaire physique, avec sa propre référence."

POURQUOI

Pourquoi séparer ARTICLE et TYPE_ARTICLE : un ordinateur précis doit être suivi individuellement (qui l'a, quel est son historique), alors qu'une cartouche d'encre n'a besoin que d'un comptage global.

Étape 4 — Architecture technique

2 min

```
React (Mois 2/3)  →  API REST (DRF)  →  Django  →  Django ORM  →  PostgreSQL
```

🗣 CE QUE JE DIS

"Django pour le backend et l'ORM, Django REST Framework pour l'API, PostgreSQL pour les contraintes d'intégrité robustes attendues d'un vrai système de gestion."

Fichier à ouvrir : `backend/inventaire/services.py`

POURQUOI

Toute la logique métier sensible est centralisée ici — jamais dans les vues ni dans l'admin directement — pour qu'une seule règle s'applique partout, quel que soit le point d'entrée (API, admin, futur import).

🐞 BUG TROUVÉ EN TEST RÉEL ET CORRIGÉ

Bug trouvé en testant l'admin, pas seulement l'API : ajouter une ligne d'entrée depuis l'admin créait la ligne mais aucun Article ni Mouvement — l'admin, par défaut, écrivait directement en base sans passer par `services.py`. Corrigé en interceptant la sauvegarde (`save_formset`) pour forcer le passage par la même logique que l'API.

🐞 BUG TROUVÉ EN TEST RÉEL ET CORRIGÉ

Deuxième bug, trouvé en testant volontairement un dépassement de stock : le code créait la Sortie en base *avant* de vérifier le stock, puis tentait de la supprimer si le blocage se déclenchait — cette suppression échouait et laissait une trace corrompue. Corrigé : le stock est maintenant vérifié *avant* toute écriture, dans une seule transaction ; si ça dépasse, rien n'est jamais écrit.

QUESTION DU JURY — "Ces bugs remettent-ils en cause la fiabilité du projet ?"

RÉPONSE : Au contraire — les avoir trouvés en testant réellement (pas seulement en lisant le code) et documentés avec leur cause exacte est la preuve d'une démarche de test sérieuse. Un projet sans aucun bug documenté serait plus suspect qu'un projet qui montre comment les problèmes ont été détectés et corrigés.

Étape 5 — Démonstration : entrées

4 min

> 🗣 **CE QUE JE DIS**
> "Je démontre maintenant, en direct, ce que j'ai déjà vérifié moi-même avant cette présentation."

1. **CE QUE JE FAIS :** Admin → Categorie "Ordinateur", Type article "Ordinateur test" (gestion individuelle cochée), Entree avec une ligne de 3.
2. **CE QUE JE MONTRE :** 3 nouveaux Article créés, référence auto-générée chacun ; Type article "Ordinateur test" → quantite_stock = 3 (pas 0) ; 3 Mouvement de type entree, un par article.

> 🗣 **CE QUE JE DIS**
>
> "*Manaraka izany, entrée d'un consommable — cartouche d'encre, tsy misy Article manokana forongoina.*"

3. **CE QUE JE FAIS :** Type article "Cartouche test" (gestion individuelle décochée), Entree avec une ligne de 50.
4. **CE QUE JE MONTRE :** aucun nouvel Article ; quantite_stock = 50 ; un seul Mouvement, avec type_article et quantite=50 renseignés, article vide.

Étape 6 — Démonstration : sortie + blocage de stock

4 min, LA PLUS IMPORTANTE

🗣 **CE QUE JE DIS**

"Je crée un Bon de Sortie pour 999 cartouches, alors qu'il n'en reste que 50."

"*Raha mihoatra ny stock ny quantité demandée, dia tsy avela ny sortie.*"

👁 **CE QUE JE MONTRE**

Admin : Direction + Bon de Sortie (statut Validé) + ligne 999. Puis POST `/api/sorties/effectuer/` avec ce bon_sortie.

🗣 **CE QUE JE DIS**

Réponse attendue : erreur 400, message explicite du dépassement.

👁 **CE QUE JE MONTRE**

Vérification : Type article "Cartouche test" est toujours à 50, aucune Sortie n'a été créée dans l'admin.

🗣 **CE QUE JE DIS**

"Maintenant avec une quantité raisonnable : un 2e Bon de Sortie, 5 cartouches."

👁 **CE QUE JE MONTRE**

201, stock 50 → 45.

🗣 **CE QUE JE DIS**

"Et une sortie individualisée : 2 ordinateurs sur les 3 en stock."

👁 **CE QUE JE MONTRE**

201, stock 3 → 1. Dans Sortie, le champ articles liste exactement 2 des 3 Article créés à l'étape précédente — jamais le même article deux fois.

QUESTION DU JURY — "Comment garantissez-vous qu'un même article ne sort pas deux fois ?"

RÉPONSE : La sélection des articles disponibles exclut ceux déjà liés à une sortie précédente. Testé automatiquement (test_08_double_sortie_impossible) : tenter une 2e sortie sur les mêmes articles est bloqué.

Étape 7 — Traçabilité et désactivation d'un compte

2 min

🗣 CE QUE JE DIS

"Afaka fantarina hoe inona no nihetsika, oviana, ary iza no nampiditra an'ilay opération."

👁 CE QUE JE MONTRE

Ouvre `/api/mouvements/` : liste complète, avec type, cible (article ou type+quantité), date, utilisateur.

🐛 BUG TROUVÉ EN TEST RÉEL ET CORRIGÉ

Correction récente : le champ `actif` sur Utilisateur n'était relié à rien — un compte désactivé pouvait quand même se connecter. Django utilise en réalité un champ différent, `is_active`, pour bloquer une connexion. Les deux sont maintenant synchronisés.

👁 CE QUE JE MONTRE

Décoche "actif" sur un compte, retente un appel API avec ce compte → 403.

Étape 8 — Les tests automatisés

2 min

👁 CE QUE JE MONTRE

Terminal : `python manage.py test inventaire`

🗣 CE QUE JE DIS

"12 tests couvrent chaque règle démontrée à l'instant, plus des cas que je ne montre pas en direct : le rollback complet si une erreur survient en cours de sortie, et le fait qu'aucune écriture partielle ne reste jamais en base."

QUESTION DU JURY — "Comment avez-vous testé ?"

RÉPONSE : Deux niveaux. Des tests automatisés (Django TestCase, 12 tests, exécutables en une commande) pour la logique métier. Et des tests manuels réels — admin, API, HTTP — qui ont permis de trouver les deux bugs présentés à l'étape 4, avant qu'ils n'arrivent en soutenance.

Étape 9 — Conclusion

2 min

🗣 CE QUE JE DIS

"Pour résumer : le socle du premier mois est posé, cohérent avec le métier réel de Paositra, et testé à deux niveaux — automatisé et manuel. Les deux bugs trouvés pendant les tests réels ont été corrigés et documentés, pas cachés."

"*Efa vita ny fototra ara-teknika. Ny sisa — interface React, QR code, dashboard, workflow feno amin'ny roa sonia — dia ho an'ny volana faha-2 sy faha-3.*"

Questions du jury — réponses prêtes

Pourquoi Django et PostgreSQL ?

Framework mature avec ORM intégré ; PostgreSQL pour des contraintes d'intégrité robustes, adaptées à un vrai système de gestion.

Pourquoi séparer Article et TypeArticle ?

TypeArticle = modèle générique ; Article = exemplaire physique à suivre individuellement. Les consommables n'ont pas besoin de ce niveau de détail.

Comment empêchez-vous une sortie supérieure au stock ?

Vérification de tout le lot avant toute écriture, dans une transaction atomique ; si une ligne dépasse, rien n'est écrit, pas même la Sortie elle-même.

Pourquoi la logique métier n'est-elle pas dans les vues ?

Pour garantir qu'admin, API et futurs points d'entrée appliquent exactement la même règle — un bug trouvé (l'admin qui la contournait) l'a confirmé concrètement.

Comment gérez-vous les utilisateurs ?

Modèle personnalisé sur la base Django (mots de passe sécurisés automatiquement) ; désactivation réelle via synchronisation actif/is_active ; jamais de suppression.

Le workflow à deux signatures (DSID puis validation) est-il fait ?

Le modèle le supporte déjà (table Intervient, rôle + date), mais la vue API qui l'automatiserait est prévue pour la suite, pas encore écrite — hors périmètre du premier mois.

Quelles sont les limites actuelles ?

Pas de gestion des retours (un article sorti ne peut pas encore être réintégré) ; pas de permissions détaillées par rôle (seule la connexion est vérifiée) ; format de référence des articles temporaire, en attente de confirmation Paositra.

Comment le projet va-t-il évoluer ?

Mois 2 : frontend React connecté à l'API existante, vue du workflow à deux signatures. Mois 3 : QR codes, dashboard, gestion des retours, permissions par rôle.

Checklist avant la présentation

- ☑ PostgreSQL démarré, migrations à jour
- ☑ python manage.py test inventaire → 12/12 OK
- ☑ Serveur démarré, admin accessible
- ☑ Les 8 étapes de démo refaites une dernière fois avant de présenter
- ☑ services.py et models.py ouverts dans l'éditeur
- ☑ JOURNAL_DECISIONS.md ouvert sur la section MCD

SI QUELQUE CHOSE CASSE PENDANT LA DÉMO

- Une requête échoue : reste calme, montre `services.py` et explique la logique sur le code plutôt qu'en live.
- Le serveur ne démarre pas : montre directement les tests automatisés déjà passés (capture d'écran à préparer avant).
- Une question sans réponse certaine : "ce point n'est pas encore implémenté, voici comment je le modéliserais" — jamais improviser une fausse certitude.